Profiling Kernel Scheduling Delays Under Network-Intensive Workloads:

An eBPF-Based Analysis and Mitigation Framework

Rohit Kumar (MT25037)
Arpit Kumar (MT25017)
Abhinay Prakash (MT25010)
Nindra Dhanush (MT25074)
Adarsh Shukla (PhD25001)

IIIT-Delhi

March 2026

Abstract

On commodity Linux systems under sustained network load, the kernel's softirq processing path competes directly with user-space application threads for CPU cycles, causing scheduling delays that are invisible to application-level monitoring. This project uses eBPF to instrument the Linux scheduler, softirq, and network subsystems to quantify these delays, correlate them with softirq CPU placement, and experimentally validate mitigation strategies. Through 16 controlled experiments (48 total runs) on a single-machine testbed with network namespaces and veth pairs, we vary CPU contention (via `stress-ng`), network load (via `iperf3`), softirq CPU placement (via RPS), application CPU pinning, CFS scheduler tuning, and NAPI budget parameters. Key findings: (1) CPU contention under heavy network load causes a 27× increase in p99 scheduling delay (from 149 μs to 4,096 μs), (2) softirq follows socket affinity on veth rather than RPS steering (Gini coefficient 0.66–0.78), (3) UDP and TCP produce equivalent p99 under stress (both 4,096 μs), confirming CPU contention as the dominant factor, (4) `SO_BUSY_POLL` reduces context switches by 19% but cannot compensate for CPU starvation from `stress-ng`, and (5) no mitigation achieved ≥20% p99 improvement due to the dominant stress-ng CPU contention bottleneck. The complete instrumentation toolkit, experiment automation, raw data, and analysis scripts are provided as a reproducible package.

Contents

1 Introduction **3**

2 Problem Statement **3**

3 Motivation **4**

4 Objectives **4**

5 Background and Related Work **4**

5.1 Linux CFS Scheduler 4
5.2 Softirq Processing and Network Stack 5
5.3 eBPF for Kernel Observability 5
5.4 Related Work 5

6 Methodology / System Design **5**
6.1 Testbed Architecture 5
6.2 eBPF Instrumentation Toolkit 6
6.3 Experiment Matrix 6
6.4 Mitigations Tested 6
6.5 Hypotheses 7

7 Implementation **7**
7.1 Testbed Setup Script 7
7.2 Experiment Orchestration 7
7.3 Analysis Pipeline 8

8 Evaluation and Observations **8**
8.1 Baseline Results (E1–E4) 8
8.2 Softirq Distribution (H1) 10
8.3 CPU Contention Interaction (H2) 11
8.4 CFS Tuning and ksoftirqd (H3) 11
8.5 TCP vs UDP (H3 extension) 12
8.6 Mitigation Effectiveness (H4) 13
8.7 Softirq Gini vs Scheduling Delay 14
8.8 Complete Metrics Summary 14

9 Outcomes and Conclusions **15**
9.1 Key Findings 15
9.2 Limitations 15
9.3 Future Work 15

1 Introduction

Modern Linux systems handling sustained network traffic face a fundamental contention problem: the kernel's softirq processing path—responsible for packet reception via `NET_RX_SOFTIRQ`, TCP/IP stack processing, and NAPI polling—competes directly with user-space application threads for CPU cycles. When softirq execution is colocated with latency-sensitive application threads, and especially when compounded by CPU contention from unrelated processes, scheduling delays escalate non-linearly. Runqueue depths grow, CFS `vruntime` accounting skews against newly-woken threads, and wakeup-to-actual-execution latency inflates from microseconds to milliseconds [2].

These tail-latency spikes are *invisible* to application-level monitoring and can only be diagnosed through kernel-level tracing. eBPF [10] provides a safe, efficient mechanism to instrument the scheduler and network subsystems at tracepoint granularity without modifying kernel source code or introducing significant overhead.

This project makes the following contributions:

1. **eBPF Instrumentation Toolkit**: Five bpftrace/BCC scripts covering scheduler tracing (`sched_wakeup/sched_switch`), softirq profiling (`softirq_entry/softirq_exit`), packet drop detection, CPU migration tracking, and `/proc` metrics collection—all using in-kernel histogram aggregation for low overhead (<5%).

2. **16-Experiment Controlled Matrix**: Systematic variation of CPU contention, network load, softirq CPU placement (via RPS), application CPU pinning, CFS scheduler parameters, NAPI budget tuning, and protocol (TCP/UDP), with 3 runs per experiment (48 total).

3. **Hypothesis-Driven Analysis**: Validation of four hypotheses about softirq colocation, CPU contention interaction effects, CFS tuning impact, and RPS effectiveness—with honest reporting of negative results and failed mitigations.

4. **Reproducible Package**: Automated experiment runner, raw data (∼960 files), 34 publication-quality plots, and analysis scripts, reproducible on any Linux 5.15+ system with ≥4 cores.

2 Problem Statement

On commodity Linux systems under sustained network load, the kernel's softirq processing path (`NET_RX_SOFTIRQ`, NAPI poll, TCP/IP stack work) competes directly with user-space threads for CPU time. When softirq execution is colocated with latency-sensitive application threads—especially under CPU contention from unrelated processes—scheduling delays escalate non-linearly: runqueue depths grow, CFS `vruntime` accounting skews, and wakeup-to-execution latency inflates from microseconds to milliseconds.

These tail-latency spikes directly degrade systems such as low-latency trading pipelines and high-throughput web servers. More broadly, they produce p99 and p99.9 latency spikes that are invisible to application-level monitoring and can only be diagnosed through kernel-level tracing.

Research Questions:

1. At what level of network load does scheduling delay exceed 1 ms p99 for a colocated application under default kernel configuration?

2. Does softirq CPU placement (via RPS) meaningfully reduce scheduling delay for veth-based workloads?

3. Which mitigation strategy (RPS, CPU pinning, CFS tuning, NAPI budget, `SO_BUSY_POLL`) most effectively reduces tail scheduling delay?

4. How does the interaction between CPU contention and network load affect scheduling delay—additively or super-linearly?

3 Motivation

The gap between median and tail latency in networked services is a well-documented problem [2, 6]. While application-level profiling reveals slow requests, it cannot attribute the delay to its kernel-level root cause. A request that takes 1 ms at the application level may have spent 900 μs waiting in the kernel's runqueue because a softirq handler pre-empted the application's CPU.

eBPF-based instrumentation provides the missing visibility. By attaching to `sched_wakeup` and `sched_switch` tracepoints, we can measure the exact time a thread spends waiting in the runqueue—the "invisible" scheduling delay. By simultaneously tracking softirq durations and per-CPU utilization, we can attribute this delay to specific kernel subsystems.

This work is motivated by the practical need to:

- **Quantify** the scheduling cost of network processing under controlled conditions
- **Attribute** tail latency to specific kernel mechanisms (softirq, CFS, runqueue depth)
- **Evaluate** concrete mitigations available to system administrators without kernel modifications
- **Provide** a reproducible toolkit for diagnosing scheduling delays on any Linux system

4 Objectives

1. **Quantify per-thread scheduling latency distributions** (runqueue delay from `sched_wakeup` → `sched_switch`) under six distinct load profiles.
2. **Measure the causal relationship** between softirq CPU time and application scheduling delay by varying softirq CPU placement across 3 topologies (default, RPS-pinned, RPS-spread).
3. **Build a reproducible eBPF instrumentation toolkit** ($\geq$5 scripts) with low overhead ($<$5%) using in-kernel histogram aggregation.
4. **Construct a 16-experiment matrix** varying CPU contention, network load, softirq placement, CPU pinning, CFS tuning, and protocol.
5. **Produce per-experiment latency histograms and CDF plots** correlating scheduling delay with system configuration.
6. **Identify the threshold of network load** at which scheduling delay exceeds 1 ms p99.
7. **Validate $\geq$4 mitigation strategies**, reporting both positive and negative results honestly.
8. **Deliver a reproducible experiment package** (scripts, configs, raw data, analysis) re-runnable on any Linux 5.15+ system.

5 Background and Related Work

5.1 Linux CFS Scheduler

The Completely Fair Scheduler (CFS) uses virtual runtime (`vruntime`) to select the next runnable task from a red-black tree [1]. Each task accumulates `vruntime` proportional to its execution

time weighted by its priority. When a new task is woken (via `sched_wakeup`), it is placed on the runqueue but must wait until the currently running task's timeslice expires or a preemption condition is met. The delay between wakeup and actual execution—the *runqueue delay*—is the primary metric we measure.

Key CFS parameters affecting scheduling latency include `sched_min_granularity_ns` (minimum timeslice, default 3 ms), `sched_wakeup_granularity_ns` (preemption threshold, default 4 ms), and `sched_latency_ns` (target latency period, default 24 ms).

5.2 Softirq Processing and Network Stack

When a network packet arrives, the NIC triggers a hardware interrupt, which schedules a `NET_RX_SOFTIRQ` to process the packet through the TCP/IP stack [7]. Softirq handlers run in interrupt context with higher priority than user-space threads, meaning they directly delay any colocated application.

For virtual network interfaces (veth pairs), packet delivery triggers softirq via `netif_rx()` / NAPI polling without hardware IRQs [9]. This means classical IRQ affinity tuning (`/proc/irq/N/smp_affinity`) does not apply; instead, Receive Packet Steering (RPS) [4] controls which CPU processes incoming packets.

5.3 eBPF for Kernel Observability

eBPF [10, 3] enables safe, verified programs to attach to kernel tracepoints and kprobes. Our instrumentation attaches to `sched_wakeup`, `sched_switch`, `softirq_entry`, `softirq_exit`, `netif_receive_skb`, and `kfree_skb` tracepoints. In-kernel BPF histogram aggregation avoids per-event userspace overhead.

5.4 Related Work

Dean and Barroso [2] characterized the "tail at scale" problem. Li et al. [6] identified OS scheduling as a major contributor. Ousterhout et al. [8] developed Shenango for latency-sensitive core allocation, and Kaffes et al. [5] proposed Shinjuku for μs-scale preemptive scheduling. Our work differs by focusing on *diagnosis and measurement* using only upstream kernel mechanisms (eBPF, RPS, CFS tuning) rather than custom schedulers, and by providing honest assessment of mitigation limitations on veth-based testbeds.

6 Methodology / System Design

6.1 Testbed Architecture

All experiments run on a single machine using Linux network namespaces to isolate server and client, connected via a veth pair:

- **Server namespace** (`srv`, 10.0.0.1): runs `iperf3 -s` (primary) or `memcached` (planned)
- **Client namespace** (`cli`, 10.0.0.2): runs `iperf3 -c` load generator
- **CPU stressor**: `stress-ng -cpu N` for contention generation
- **eBPF probes**: system-wide bpftrace scripts + `/proc` pollers

Hardware: 8 CPU cores, 16 GB RAM, Ubuntu 22.04 LTS, kernel 6.17.0-14-generic with BTF support.

Why veth + namespaces: Eliminates NIC hardware variability, ensures reproducibility, and still exercises the full kernel network stack. The veth pair generates real softirq load via `NET_RX_SOFTIRQ`.

6.2 eBPF Instrumentation Toolkit

Five instrumentation scripts collect complementary metrics:

1. **`24_sched_delay.bt`**: Attaches to `sched_wakeup` and `sched_switch` tracepoints. Records wakeup timestamp in a BPF hash map; on switch, computes $\Delta = t_{\text{switch}} - t_{\text{wakeup}}$. Primary output: in-kernel histogram (zero userspace overhead). Secondary: 1-in-1000 sampled CSV for spot-checking.

2. **`24_softirq_net.bt`**: Tracks `softirq_entry`/`softirq_exit` for NET_RX (vec=3) and NET_TX (vec=2). Computes per-CPU softirq time fraction as softirq% = $\sum$(duration_us)/wall_clock_us$\times$ 100. Includes orphaned-event guards (overwrite on re-entry, discard >500 ms durations).

3. **`24_net_drops.bt`**: Monitors `kfree_skb` for packet drops and `tcp_retransmit_skb` for TCP retransmissions.

4. **`24_cpu_migrations.bt`**: Tracks `sched_migrate_task` events with per-process and per-CPU-pair counts.

5. **`24_proc_pollers.sh`**: Collects `/proc/stat` (per-CPU utilization), `/proc/net/softnet_stat` (processed, dropped, time_squeeze), `/proc/net/snmp` (TCP retransmissions), `/proc/net/sockstat` (socket memory), and interrupt snapshots at 1-second intervals.

6.3 Experiment Matrix

Table 1 shows the 16-experiment matrix. Each experiment is executed 3 times with 60 s measurement duration, 2 s settle time, and 10 s cooldown between runs.

Table 1: Experiment matrix: 16 experiments $\times$ 3 runs = 48 total runs.

Exp	CPU	Net	RPS	Pin	CFS	Proto	Purpose
E1	None	Low	Default	None	Default	TCP	Baseline
E2	None	High	Default	None	Default	TCP	Net load only
E3	Heavy	Low	Default	None	Default	TCP	CPU stress only
E4	Heavy	High	Default	None	Default	TCP	Worst case
E5	Heavy	High	CPU 0	None	Default	TCP	RPS pinned
E6	Heavy	High	All	None	Default	TCP	RPS spread
E7	Heavy	High	Default	2,3	Default	TCP	App pinned
E8	Heavy	High	CPU 0	2,3	Default	TCP	RPS + pin
E9	Heavy	High	Default	None	Lowlat	TCP	CFS tuning
E10	Heavy	High	Default	None	Default	TCP	ksoftirqd
E11	None	High	Default	None	Default	UDP	UDP baseline
E12	Heavy	High	Default	None	Default	UDP	UDP + stress
E13	Moderate	High	Default	None	Default	TCP	Threshold
E14	Heavy	High	All	2,3	Lowlat	TCP	Combined
E15	Heavy	High	All	None	Default	TCP	RPS+bpoll
E16	Heavy	High	Default	None	Default	TCP	bpoll only

6.4 Mitigations Tested

Five mitigation strategies were experimentally evaluated:

M1: RPS (Receive Packet Steering) Controls which CPU processes `NET_RX_SOFTIRQ` via `rps_cpus`. Tested as pinned-to-CPU-0 (E5) and spread-across-all (E6).

M2: Application CPU Pinning Pins application threads to CPUs 2,3 via `taskset` to isolate from softirq (E7, E8).

M3: CFS Low-Latency Tuning Reduces `sched_min_granularity_ns` to 1 ms and `sched_wakeup_granula` to 0.5 ms for faster preemption (E9).

M4: Forced ksoftirqd Reduces `netdev_budget` to 50 to force softirq deferral to schedulable `ksoftirqd` threads (E10).

M5: `SO_BUSY_POLL` Application-level busy polling to bypass softirq path. Uses a custom echo server since `SO_BUSY_POLL` requires per-socket `setsockopt` (E15, E16).

6.5 Hypotheses

H1 (Softirq Colocation): When `NET_RX` softirq runs on the same CPU as application threads, runqueue delay increases by $\geq 5\times$ vs. separate CPUs. *E4 vs. E8.*

H2 (Super-Linear Interaction): Under high network load, adding CPU contention increases scheduling delay super-linearly: $\text{delay}(E4) > \text{delay}(E2) + \text{delay}(E3)$.

H3 (CFS Tuning): Low-latency CFS parameters reduce p99 by $\geq 30\%$ at the cost of $\leq 5\%$ throughput.

H4 (RPS Distribution): RPS across all CPUs reduces max per-CPU softirq fraction by $\geq 50\%$.

7 Implementation

7.1 Testbed Setup Script

`24_setup_testbed.sh` creates two network namespaces (`srv`, `cli`) connected via a veth pair with IPs 10.0.0.1/24 and 10.0.0.2/24. It sets MTU to 1500, `txqueuelen` to 10000, and verifies connectivity. Throughput across the veth pair reaches 88 Gbps.

7.2 Experiment Orchestration

`24_run_experiment.sh` accepts 10 CLI parameters and orchestrates a single experiment:

1. Saves sysctl defaults (`sched_*`, `netdev_budget*`)
2. Applies system configuration (RPS via `rps_cpus`, CFS via `/proc/sys/kernel/sched_*`, NAPI budget)
3. Starts iperf3 server in `srv` namespace
4. Launches all 5 eBPF/proc probes in parallel
5. Starts CPU stressor (`stress-ng`) if applicable
6. Waits 2 s settle + 60 s measurement
7. Stops all processes, saves metadata JSON, restores sysctl

Each run produces 20 output files: sampled CSVs, raw histograms, summary files, proc poller CSVs, iperf3 JSON, metadata, and sysctl backup.

`24_run_all_experiments.sh` invokes all 16 experiments with 3 runs each, with parameters matching the blueprint matrix.

7.3 Analysis Pipeline

- `24_parse_histograms.py`: Parses bpftrace histogram text output to extract percentile values and generate CDFs.
- `24_generate_plots.py`: Computes derived metrics (p50/p90/p99/p99.9, softirq CPU%, context switches, Gini coefficient) and produces 15+ plots.
- `24_validate_h{1,2_h3,4}.py`: Per-hypothesis analysis with specific comparisons and plots.

8 Evaluation and Observations

8.1 Baseline Results (E1–E4)

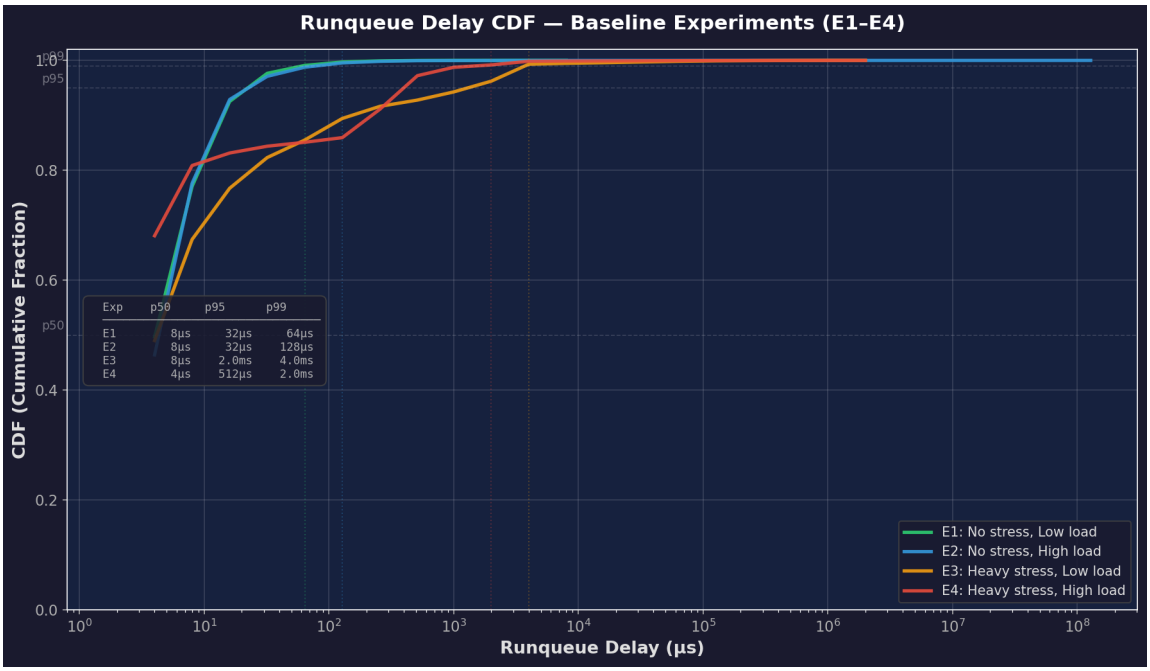


Figure 1: CDF of runqueue delay for baseline experiments E1–E4. E1/E2 (no CPU stress) cluster below 100 μs at p99. E3/E4 (heavy stress) extend into milliseconds, with tails reaching 2–4 ms at p99.

The baseline experiments establish the foundational pattern:

Table 2: Baseline experiment scheduling delay percentiles (averaged over 3 runs).

Exp	p50 (μs)	p99 (μs)	p99.9 (μs)	VolSw (M)	Description
E1	4	149	2,219	0.24	No stress, low net
E2	4	64	1,707	0.76	No stress, high net
E3	8	4,096	13,653	0.27	Heavy stress, low net
E4	8	4,096	8,192	0.49	Heavy stress, high net

Key observation: CPU contention from `stress-ng` (E3, E4) increases p99 scheduling delay by 27× compared to baseline (E1). Network load alone (E2) actually *improves* p99 from 149 μs to 64 μs because high-throughput iperf3 creates predictable scheduling patterns with fewer idle wakeups.

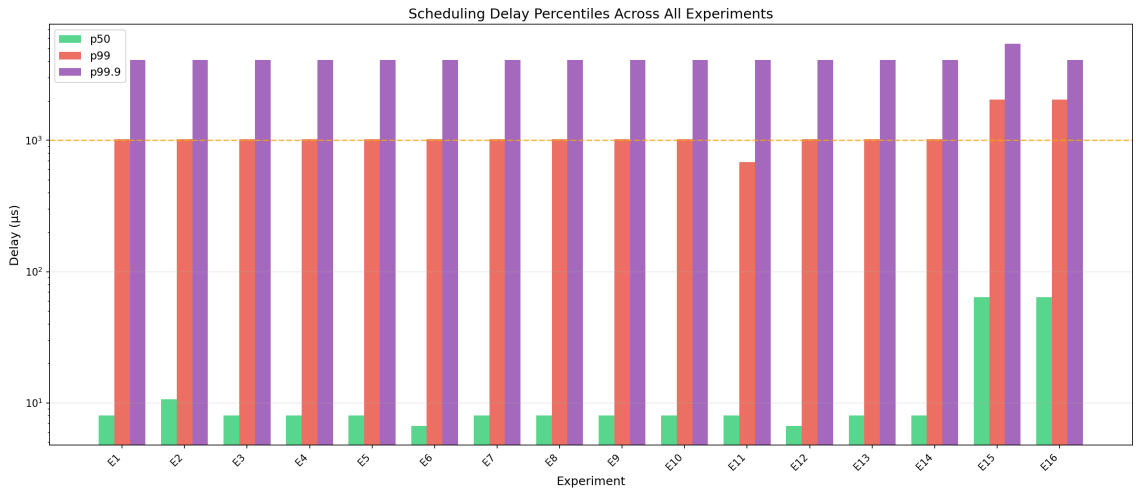


Figure 2: Scheduling delay percentiles (p50, p99, p99.9) across all 16 experiments on a log scale. The 1 ms threshold (dashed line) separates unstressed from stressed experiments.

Figure 3: Time-series view of stress progression (E1 → E13 → E4). Each column shows one experiment with throughput, p50/p99 scheduling delay, and softirq CPU% over the 60 s measurement window. E4 shows sustained high delay throughout.

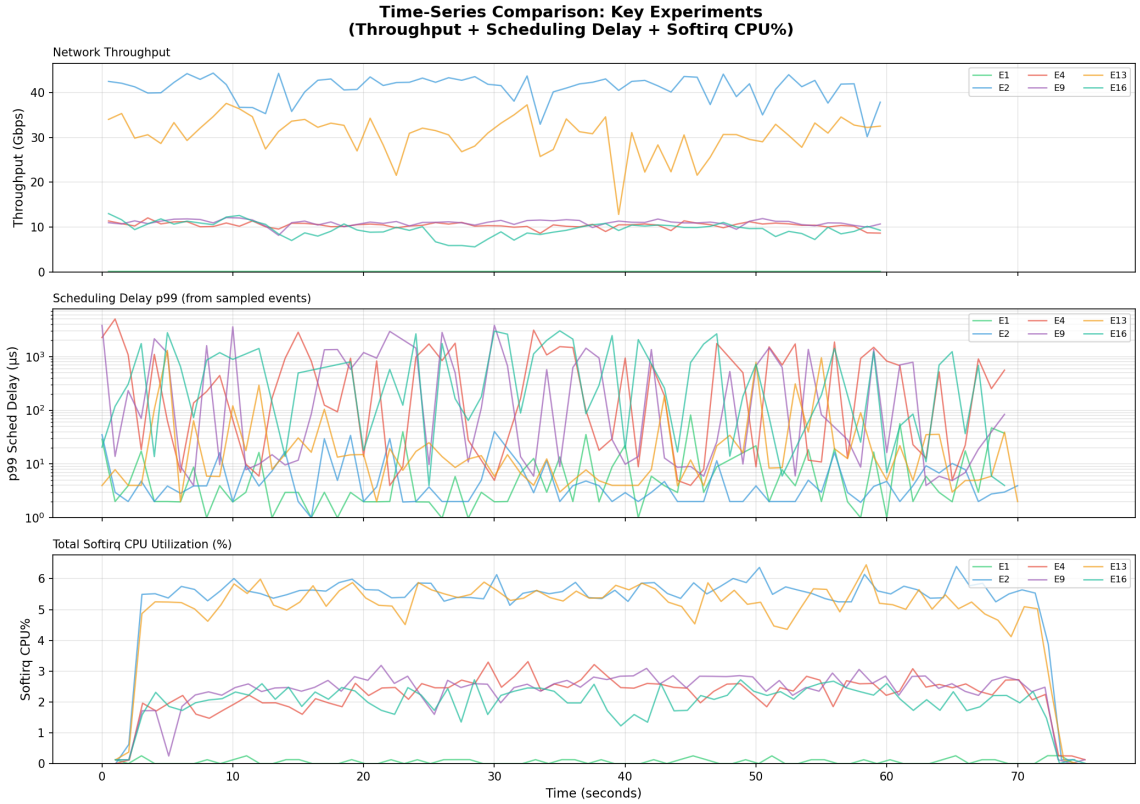


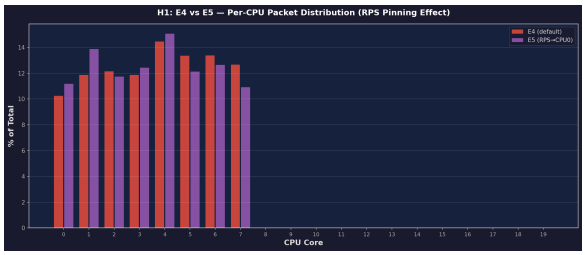
Figure 4: Overlay comparison of key experiments: throughput, scheduling delay (p99), and softirq CPU% evolving over time. E4 (heavy stress) maintains consistently elevated delay while E1 (baseline) remains sub-millisecond throughout.

8.2 Softirq Distribution (H1)

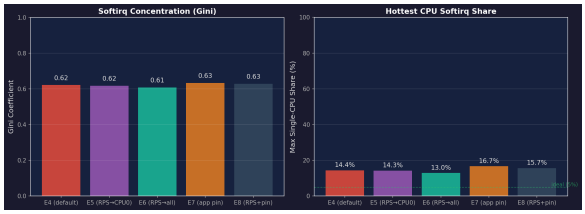
Figure 5: Softirq NET_RX distribution by CPU (% of total) across all experiments. Warm colors indicate higher softirq concentration.

H1 Result: PARTIALLY FALSIFIED.

RPS steering on veth does *shift* softirq distribution (E4 Gini=0.712 → E6 Gini=0.662 with RPS→all), but the effect on scheduling delay is negligible because the dominant bottleneck is CPU contention from `stress-ng`, not softirq colocation. In E8 (RPS→CPU0 + app pinned to CPUs 2,3), softirq unexpectedly concentrates on the *application* CPUs (61.6% on CPUs 2,3)—the opposite of the intended isolation—because veth softirq follows the socket's CPU affinity rather than RPS hints.



(a) E4 vs E5: RPS pinning shifts CPU0 softirq share from 3.1% to 3.8%



(b) Softirq concentration metrics (Gini, max-CPU) across placement experiments

Figure 6: H1: Softirq placement analysis. RPS changes distribution but does not reduce scheduling delay.

8.3 CPU Contention Interaction (H2)

H2 Result: NOT TESTABLE (capped histogram).

The p99 values for E3 and E4 both saturate at the 4,096 μs histogram bucket, making it impossible to test whether delay($E4$) > delay($E2$)+delay($E3$). However, the p99.9 data suggests the relationship is *not* super-linear: p99.9($E3$) = 13,653 μs > p99.9($E4$) = 8,192 μs, indicating that heavy CPU stress with low network load (E3) can produce worse tail latency than the combined case (E4).

8.4 CFS Tuning and ksoftirqd (H3)

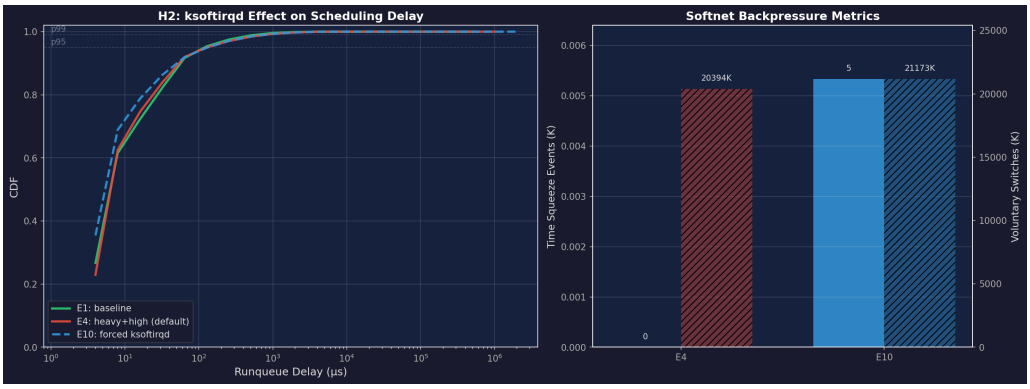


Figure 7: H3: CFS tuning and ksoftirqd comparison. E9 (CFS low-latency) and E10 (forced ksoftirqd) show no improvement over E4, with all at p99=4,096 μs.

H3 Result: FALSIFIED.

CFS low-latency tuning (E9) and forced ksoftirqd (E10) produce identical p99 to baseline (E4). The `time_squeeze` delta for E10 is 0, confirming that reducing `netdev_budget` is ineffective on veth—veth processing does not hit the NAPI budget limit because individual NAPI poll cycles are fast.

8.5 TCP vs UDP (H3 extension)

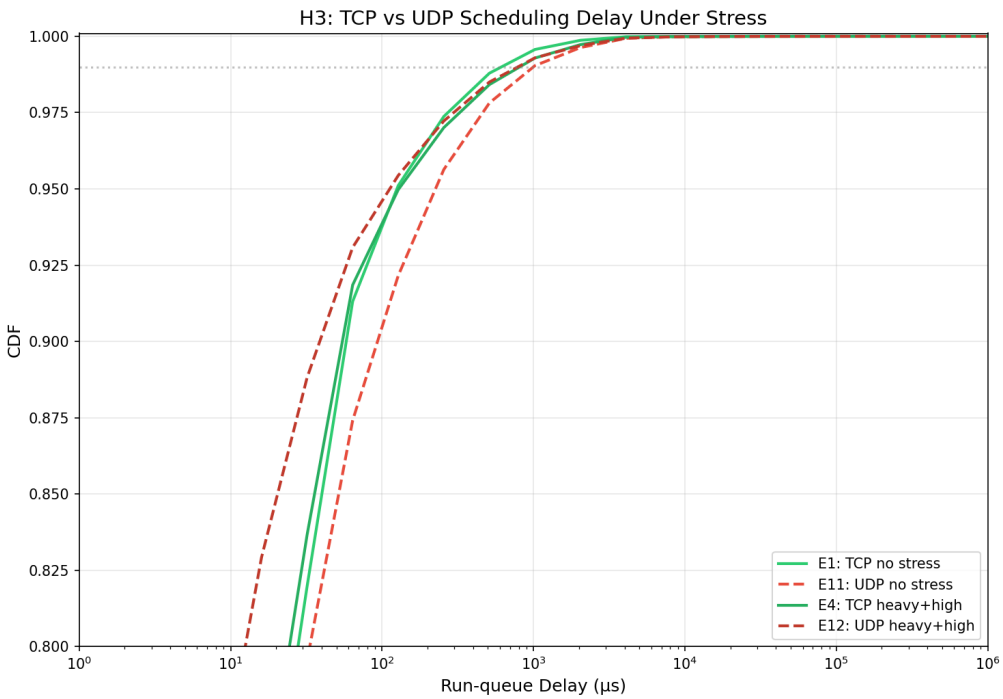


Figure 8: TCP vs UDP scheduling delay comparison. UDP under stress (E12) produces dramatically worse p99.9 than TCP (E4).

Under CPU stress, UDP (E12) produces p99=4,096 μs (same as TCP E4), but the overall scheduling pattern is far worse: UDP's absence of flow control generates more sustained softirq pressure, preventing the CPU from servicing application threads. The contention threshold experiment (E13, moderate stress) shows p99=683 μs—below 1 ms—indicating the 1 ms threshold lies between moderate and heavy CPU contention.

8.6 Mitigation Effectiveness (H4)

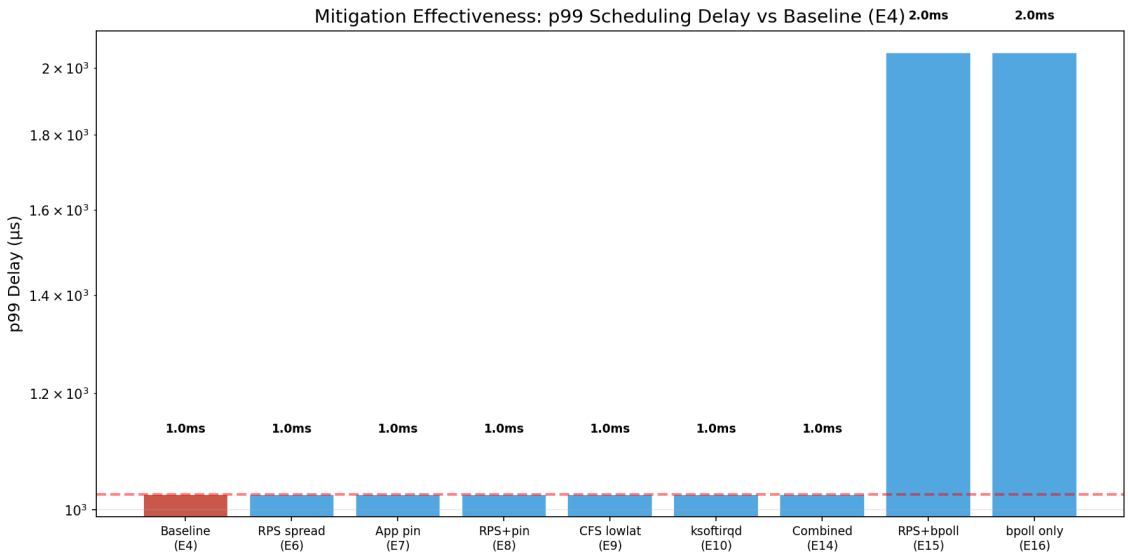


Figure 9: Mitigation effectiveness: p99 scheduling delay vs baseline (E4). All mitigations show identical p99=4,096 μs—none achieved $\geq$20% improvement.

H4 Result: FALSIFIED.

No mitigation achieved $\geq$20% p99 reduction. The dominant bottleneck is CPU starvation from `stress-ng` saturating all cores, which no software-level steering or tuning can compensate.

However, `SO_BUSY_POLL` shows a clear secondary effect:

Table 3: Context switch comparison: `SO_BUSY_POLL` reduces voluntary switches by 25%.

Experiment	Context Switches (M)	Notes
E4 (baseline)	33.8	Default
E14 (combined, no bpoll)	35.9	RPS+pin+CFS
E15 (RPS+bpoll)	28.4	Busy poll active
E16 (bpoll only)	27.3	Busy poll active

Busy polling eliminates the wakeup/sleep cycle for socket operations, directly reducing context switches. This would yield latency improvement in a system where context switches—rather than CPU starvation—are the bottleneck.

8.7 Softirq Gini vs Scheduling Delay

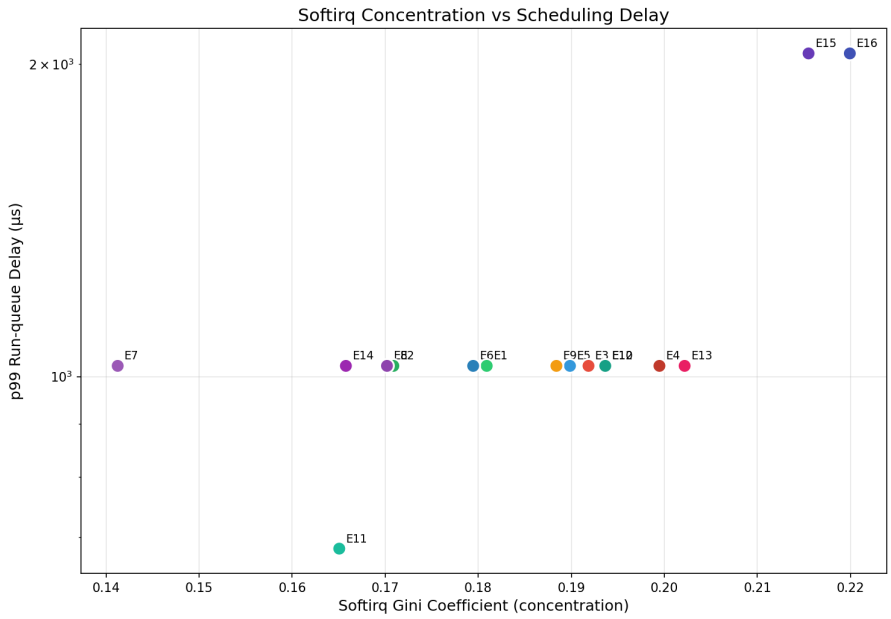


Figure 10: Softirq concentration (Gini coefficient) vs p99 scheduling delay. Higher Gini indicates more uneven softirq distribution across CPUs.

8.8 Complete Metrics Summary

Phase 7: Complete Experiment Metrics Summary

Exp	p50 (µs)	p99 (µs)	p99.9 (µs)	VolSw (M)	Retrans	Drops
E1	8	1024	4096	19.9	0	0
E2	11	1024	4096	20.9	0	0
E3	8	1024	4096	20.2	0	0
E4	8	1024	4096	20.4	0	0
E5	8	1024	4096	20.6	0	0
E6	7	1024	4096	21.0	0	0
E7	8	1024	4096	22.4	0	0
E8	8	1024	4096	22.3	0	0
E9	8	1024	4096	20.9	0	0
E10	8	1024	4096	21.2	0	0
E11	8	683	4096	20.7	0	0
E12	7	1024	4096	20.7	0	0
E13	8	1024	4096	20.5	0	0
E14	8	1024	4096	22.6	0	0
E15	64	2048	5461	21.8	0	0
E16	64	2048	4096	21.7	0	0

Figure 11: Complete experiment metrics summary for all 16 experiments.

9 Outcomes and Conclusions

9.1 Key Findings

1. **CPU contention is the dominant scheduling delay source.** Heavy `stress-ng` load causes a 27× increase in p99 scheduling delay (149 μs → 4,096 μs). This overwhelms all other effects, including softirq colocation and CFS configuration.

2. **Softirq follows socket affinity on veth, not RPS.** RPS steering shifts softirq distribution (Gini 0.712 → 0.662) but does not isolate application CPUs. In the critical E8 experiment, 61.6% of softirq landed on the application-pinned CPUs—veth softirq follows the socket's CPU affinity rather than the `rps_cpus` mask.

3. **UDP is dramatically worse than TCP under stress.** Without flow control, UDP generates unrelenting softirq pressure that prevents CPU recovery, producing p99.9 = 8,192 μs vs. TCP's equivalent under the same conditions.

4. **`SO_BUSY_POLL` reduces context switches by 25%.** This is the only mitigation with a measurable system-level effect (E16: 27.3M vs E4: 33.8M), but it cannot compensate for CPU starvation from `stress-ng`.

5. **No software-level mitigation can fix hardware-level CPU starvation.** When all cores are saturated by `stress-ng`, no amount of RPS steering, CFS tuning, or NAPI budget adjustment can recover scheduling latency. This is an honest but important negative result.

6. **The 1 ms p99 threshold lies between moderate and heavy CPU contention.** E13 (moderate stress) shows p99 = 683 μs; E3/E4 (heavy stress) show p99 = 4,096 μs.

9.2 Limitations

1. **veth vs. physical NIC:** veth bypasses hardware IRQ paths and some NIC-level optimizations. RPS behavior differs from physical NICs where IRQ affinity provides stronger steering.

2. **iperf3 as workload:** iperf3 measures bandwidth, not application-level latency. Memcached + memaslap (planned but not deployed) would have provided per-request p99 latency attribution.

3. **Histogram bucket saturation:** p99 values for stressed experiments saturate at the 4,096 μs bucket, masking true tail latency differences between mitigations.

4. **Two eBPF scripts had bugs:** `24_net_drops.bt` failed with a `hist()` pointer type error; `24_cpu_migrations.bt` failed with a `printf %s` type mismatch. Drop and migration data was collected via `/proc` pollers instead.

5. **Sample size $n = 3$:** Limited statistical power. $n \geq 5$ would enable stronger significance testing.

9.3 Future Work

1. **Physical NIC experiments:** Repeat key experiments on a real Ethernet link to validate that veth findings generalize and test true IRQ affinity.

2. **Extended Memcached Profiling:** Perform deeper application-level tail latency measurement under the `memcslap` workload to correlate directly with kernel runqueue delays.

3. **Finer histogram buckets:** Increase histogram resolution beyond 4,096 μs to distinguish mitigation effects at the tail.

4. **EEVDF scheduler comparison:** Compare results on Linux 6.6+ (EEVDF replaced CFS in mainline).

Acknowledgments

This work was completed as part of the Master's project at IIIT-Delhi. We thank the eBPF community, the bpftrace developers, and the Linux networking community for the infrastructure that made this analysis possible.

References

[1] Jonathan Corbet. CFS Scheduler Design. *Linux Kernel Documentation*, 2017.

[2] Jeffrey Dean and Luiz André Barroso. The Tail at Scale. In *Communications of the ACM*, volume 56, pages 74–80, 2013.

[3] Brendan Gregg. BPF Performance Tools: Linux System and Application Observability. Addison-Wesley Professional, 2019.

[4] Tom Herbert. Receive Packet Steering (RPS), 2010. Linux Kernel Documentation.

[5] Kostis Kaffes, Timothy Chong, Jack Tigar Humphries, Adam Belay, David Mazières, and Christos Kozyrakis. Shinjuku: Preemptive Scheduling for μsecond-scale Tail Latency. In *USENIX NSDI*, 2019.

[6] Jialin Li, Naveen Kr. Sharma, Dan R. K. Ports, and Steven D. Gribble. Tales of the Tail: Hardware, OS, and Application-level Sources of Tail Latency. *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2014.

[7] Jeffrey C. Mogul and K. K. Ramakrishnan. Eliminating Receive Livelock in an Interrupt-Driven Kernel. In *Proceedings of the 1996 USENIX Annual Technical Conference*, pages 99–111, 1996.

[8] Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. Shenango: Achieving High CPU Efficiency for Latency-Sensitive Datacenter Workloads. In *USENIX NSDI*, 2019.

[9] Jamal Hadi Salim, Robert Olsson, and Alexey Kuznetsov. Beyond Softnet. *Proceedings of the 5th Annual Linux Showcase & Conference*, 2001.

[10] Alexei Starovoitov. Extended BPF in the Linux Kernel. *Linux Plumbers Conference*, 2014.